

第23-24课 PDF解析整理版

来源：飞书「ChatBI 项目系统蓝图」解析整理版。原课件 PDF 附件无下载权限，本文为文档正文导出。

第 23-24课 PDF 解析整理版

第23课 + 第24课：Plan-and-Execute Agent 构建与执行管理

课程总结与关系

演进关系：

- 第22课解决“拆什么”（子任务分解）。
- 第23课解决“怎么按顺序执行”（计划生成与顺序调度）。
- 第24课解决“执行过程中怎么管中间结果”（状态、重试、依赖跳过、结果引用）。

三者共同构成一个可稳定运行的 Plan-and-Execute Agent 雏形。

第23课：构建Plan-and-Execute Agent框架

课程定位与目标

课程定位

第22课已经把复杂问题拆成了结构化子任务，但系统此时拿到的仍然只是一份“待办清单”，还不是可以直接运行的执行链路。

第23课要解决的问题，是把这份清单继续推进成一个最小可运行的Agent骨架，让系统第一次具备“先规划，再逐步执行”的能力。

本节课目标

- 理解为什么子任务列表不等于执行计划
- 理清Planner、Executor、Summarizer这3个角色的边界
- 实现agent_planner.py，跑通顺序执行版Agent主链路
- 用真实命令验证计划生成、依赖传递和结果汇总

开场引入

第22课已经把“最近三个月利润为什么下降”拆成3个子任务：

- 先看利润趋势
- 再拆收入与成本变化
- 最后定位利润下滑最严重的区域

看起来已经很完整了，但系统还是不能直接运行。

真正缺的不是“任务有没有拆开”，而是下面这3件事：

- 谁先执行
- 每一步具体问什么
- 上一步结果怎么传给下一步

拆解结果回答的是“要做哪些事”，执行计划回答的是“下一秒先做哪件事、拿什么结果继续往下做”。

一、为什么有了子任务列表还不够

先看第22课输出和第23课执行计划的区别：

如果只有subtasks，系统会遇到两个现实问题：

- task_2依赖task_1，但代码层还不知道该什么时候执行
- “拆解收入与成本变化”只是一个标题，现有Text2SQL主链路并不能直接理解它该查什么

所以第23课要补的，是一个编排层：

从这一课开始，系统不再只会“把一句话翻成一条SQL”，而是开始具备“围绕目标组织多个查询动作”的能力。

二、Plan-and-Execute Agent的3个角色

这一课的Agent只保留最核心的3个角色。

2.1 Planner：把子任务翻译成执行步骤

Planner的职责不是重新思考业务问题，而是把第22课已经产出的子任务列表，转换成系统可直接运行的steps。

- 给每一步分配step_id
- 把task_id依赖映射成step_id依赖

- 补齐每一步的执行问题、关注指标和预期输出

2.2 Executor：逐步调用已有Text2SQL系统

Executor不负责拆任务，也不负责写最终报告，它只做一件事：按顺序执行计划中的每一步，并把结果记下来。

2.3 Summarizer：把分散结果收成统一摘要

Summarizer在这一课里先做轻量版：

- 统计完成了几步
- 收集每一步的关键发现
- 输出一段可被后续模块继续消费的统一摘要

这3个角色的关系如下：

第23课先做“能规划、能执行、能收口”的最小闭环：中间结果存储、失败重试、动态改计划，统一留到后面再展开。

三、计划生成

3.1 计划数据结构

第一步，不是先写执行器，而是先把“计划长什么样”定义清楚。

在 code/chatbi_mvp/agent_planner.py 里，核心计划结构是 PlanStep：

```
class PlanStep(BaseModel):
    step_id: str
    task_id: str
    step_name: str
    task_type: str
    action: str = "text2sql"
    question: str
    description: str
    depends_on: list[str] = Field(default_factory=list)
    metrics: list[str] = Field(default_factory=list)
    dimensions: list[str] = Field(default_factory=list)
    expected_output: str
```

这个结构的关键不在“字段多不多”，而在于它把第22课的抽象子任务，变成了第23课可执行的步骤对象。

- task_id 负责和拆解结果对齐
- step_id 负责执行顺序

- question 负责真正喂给现有 Text2SQL 系统
- expected_output 负责提醒这一步应该产出什么

Planner 做的事情，本质上就是一次“翻译”：

3.2 计划生成关键代码

```
task_to_step = {
    task.task_id: f"step_{index}"
    for index, task in enumerate(plan.subtasks, start=1)
}

steps.append(
    PlanStep(
        step_id=f"step_{index}",
        task_id=task.task_id,
        step_name=task.task_name,
        task_type=task.task_type,
        question=self._build_step_question(task),
        depends_on=[task_to_step[dep] for dep in task.depends_on],
        metrics=task.metrics,
        dimensions=task.dimensions,
        expected_output=self._build_expected_output(task),
    )
)
```

这里最重要的动作只有两个：

- 把 task_1 -> task_2 这种依赖，翻译成 step_1 -> step_2
- 把“拆解收入与成本变化”这种标题，展开成真正可执行的问题文本

Planner 不是新的 SQL 生成器，而是把“子任务描述”加工成“执行器能接住的输入”。

四、Step Executor 如何串起已有主链路

Step Executor 直接复用前面课程中已经写好的 ChatBISystem.run()。

构造依赖上下文（中间结果管理：“文本化上下文拼接”）



拼接上下文，组装问题



单步执行，拿到执行结果

```
for step in plan.steps:
    dependency_context = self._build_dependency_context(
        step.depends_on, results_by_step,
    )
    composed_question = self._compose_question(step.question, dependency_context)
    raw_result = self.step_runner(composed_question)
```

例如第二步真正执行前，系统会自动拼接前置结果（前一步结果会影响后一步问题）：

请执行子任务：拆解收入与成本变化。

前置步骤关键结果如下，请在本次查询中延续这些上下文：

- 明确利润下降的时间范围与下降幅度：2026-04的利润最低，为-1886156.00000000。

为了避免结果太散，单步执行器Executor 还会把单步输出统一整理成 StepExecutionResult：

- 是否成功 success
- 实际 SQL sql
- 结果列 columns
- 结果行 rows
- 文本摘要 formatted

这节课通过一种简单的“中间结果传递方案”，让系统第一次具备“沿着依赖链连续执行”的能力。

五、把 Planner、Executor、Summarizer 收口为统一入口

如果这3个角色各写各的，系统还是很难调用，所以还需要一个总入口。

在 PlanAndExecuteAgent.run() 里，整条链路被压成了4步：

```
decomposition = decomposition_override or self.decomposer.decompose(user_question)
plan = self.planner.build_plan(user_question, decomposition)
step_results = self.executor.execute_plan(plan)
summary = self.summarizer.summarize(user_question, plan, step_results)
```

这段代码对应的就是本课最核心的主链路：

到这里，系统虽然还没有“自动重试”“动态改计划”“结构化中间结果仓库”，但已经第一次具备了Agent的骨架。

六、运行准备与真实验证

6.1 运行环境

本节代码目录：code/chatbi_mvp

统一运行方式：

```
cd code/chatbi_mvp
uv sync
```

6.2 单元测试

单元测试完成3件事，分别验证Planner、Executor、Summarizer可以单独正常运行。

```
uv run pytest tests/test_agent_planner.py
```

真实输出（节选）：

```
===== test session starts =====
platform darwin -- Python 3.12.9, pytest-9.1.1, pluggy-1.6.0
collected 3 items
tests/test_agent_planner.py ... [100%]
===== 3 passed in 0.37s
```

这3个测试分别验证了：

- Planner 能否正确生成带顺序的 steps
- Executor 会不会把前置步骤结果拼到后续问题里
- Agent 总入口能否输出统一摘要

6.3 拆解与计划生成

```
uv run agent_planner.py --plan-only
```

输出结果（节选）：

- QueryDecomposer 能真实返回结构化子任务
- PlanGenerator 能把子任务真实翻译成执行步骤

```
"decomposition": {
  "question_type": "利润下降原因分析",
  "analysis_goal":
"识别最近三个月利润下降的主要原因，明确利润下降发生的时间区间、影响维度、核心驱动指标及异常业务环节，为后续经营优化提供依据。"
}
"plan": {
  "steps": [
    {
      "step_id": "step_1",
      "step_name": "确认最近三个月利润变化趋势",
      "task_type": "时间趋势分析"
    }
  ]
}
```

6.4 执行链路验证（首步）

执行阶段先只跑第1步，避免一上来把10个步骤全部堆到一起，难以定位问题：

```
uv run agent_planner.py --max-steps 1 --disable-schema-linking --disable-indicator-rag
```

某次真实输出（节选）：

```
{
  "step_results": [
    {
      "step_id": "step_1",
      "step_name": "明确利润下降的时间范围与下降幅度",
      "success": true,
      "columns": [
        "profit_month",
        "profit",
        "profit_change_amount",
        "profit_mom_growth_rate",
        "is_profit_decline",
        "is_abnormal_low_month"
      ]
    }
  ],
  "summary": {
    "completed_steps": 1,
    "failed_steps": 0
  }
}
```

同一次真实运行里，返回的结果表如下：

profit_month	profit	profit_change_amount	profit_mom_growth_rate
2026-04	-1886156.00000000	None	None
2026-05	-1088608.40000000	797547.60000000	-42.28
2026-06	0E-8	1088608.40000000	-100.00

到此为止，本节课已经真实接上了：

- 复杂问题拆解
- 执行计划生成
- 第一步真实调用 Text2SQL，数据库返回真实查询结果

七、常见误区

7.1 把 Planner 写成新的 SQL 生成器

Planner 的职责是“排计划”，不是“生成 SQL”。

如果在 Planner 里直接开始拼 SQL，本课的分层就会立刻混乱，前面已经做好的 Text2SQL 主链路也会被绕开。

7.2 每一步都直接追求最终业务结论

本课的每一步更像“取证”，不是“判案”。

前两步应该先拿到趋势和拆解结果，最后一步再把区域层面的发现补齐；如果一开始就让每一步都输出最终归因

，后续步骤反而没有存在意义。

Plan-and-Execute 的价值，不是把一句话拆成更多句，而是让每一步都成为下一步的可靠输入。

小结

第23课完成了 Agent 阶段真正意义上的第一次落地：

- 第22课解决“拆什么”
- 第23课解决“怎么按顺序执行”
- 第24课将继续解决“执行过程中怎么管中间结果”

可以把本节课的贡献压缩成下面这张图：

到这一课为止，系统已经不再只是一个“会写 SQL 的助手”，而是一个开始具备分析编排能力的 Agent 雏形。

附录

附录A：本节代码

- `code/chatbi_mvp/agent_planner.py`
- `code/chatbi_mvp/tests/test_agent_planner.py`

建议重点阅读的代码段

- `PlanStep / ExecutionPlan`：计划结构定义
 - `PlanGenerator.build_plan()`：子任务到步骤的映射逻辑
 - `StepExecutor.execute_plan()`：顺序执行与依赖上下文拼接
 - `PlanAndExecuteAgent.run()`：总入口主链路
-

第24课：多SQL执行与中间结果管理

课程定位与目标

课程定位

第23课已经让Agent第一次具备了“先规划、再执行、最后汇总”的最小闭环，但那个版本更像一个简单可跑的顺序执行器。

一旦分析任务真的进入企业场景，系统很快就会遇到3个现实问题：

1. 上一步查出来的结果，下一步到底怎么接着用
2. 某一步SQL执行失败后，是不是整条链路就直接断掉
3. 多步执行完成后，系统怎么知道哪些结果可信、哪些步骤被跳过

本节课目标

- 理解为什么多步执行不等于把3条SQL顺序跑完
 - 建立ExecutionContext视角，明确中间结果、步骤状态、结果引用分别解决什么问题
 - 掌握两种中间结果方案：内存结果集与临时表
 - 升级agent_planner.py，补齐重试、跳过和结果引用机制
-

开场引入

假设系统接到这个问题：

最近三个月利润为什么下降？

第23课的Agent已经能把它拆成3步：

1. 先看利润趋势
2. 再拆收入与成本变化
3. 最后定位利润下滑最严重的区域

问题在于，这3步虽然能按顺序执行，但系统还没有真正学会“接力”。

第一步查出了2026-05的利润最低，第二步如果拿不到这个结果，就不知道该继续围绕哪个月份展开；如果第二步SQL失败，第三步也不该装作什么都没发生继续查。

Agent真正的难点，不是“能不能生成多条SQL”，而是多条SQL之间如何传递结果、记录状态，并在失败时维持可控行为。

一、本节课边界

第23课的执行器默认做了两件事：

- 拼接前置步骤的文字摘要
- 顺序调用ChatBISystem.run()

这已经足够做最小闭环，但还缺3类企业场景里一定会遇到的信息：

- 结果引用：后续步骤要知道“上一份结果存在哪里”
- 执行状态：当前步骤是成功、失败，还是因为上游失败被跳过
- 尝试次数：一次成功，还是重试了2次才成功

第23课解决的是“怎么排队执行”，第24课解决的是“执行过程中发生了什么，以及这些信息如何被后续步骤继续消费”。

二、执行全流程分析

真正进入代码之前，先把执行器在第24课里要做的每一步画清楚。

上面这张图中最重要的5个控制点：

1. 依赖检查：先判断上游步骤是否已经失败
2. 上下文拼接：把上一步的关键结果转成后续步骤可消费的提示
3. 失败重试：同一步允许重复尝试
4. 状态落盘：每一步都要记录 completed / failed / skipped
5. 结果引用：成功结果要生成 memory://step_x 或 temp_table://tmp_step_x

本节课真正新增的，是执行器对“成功、失败、跳过、重试、引用”这几种状态的完整管理。

三、中间结果管理

3.1 中间结果要存什么

一个真正可用的执行上下文，至少要同时记录4类信息：

- 数据本身：列名、行数据、格式化摘要
- 运行元信息：SQL、尝试次数、错误信息
- 引用信息：后续步骤如何找到这份结果
- 状态信息：成功、失败、跳过

在第24课的代码里，StepExecutionResult被扩展成下面这类结构：

```
class StepExecutionResult(BaseModel):
    step_id: str
    task_id: str
    step_name: str
    success: bool
    status: Literal["completed", "failed", "skipped"] = "completed"
    attempts: int = 1
    question: str
    depends_on: list[str] = Field(default_factory=list)
    context_used: str = ""
    storage_backend: str | None = None
    result_reference: str | None = None
    sql: str | None = None
    columns: list[str] = Field(default_factory=list)
    rows: list[dict[str, Any]] = Field(default_factory=list)
    formatted: str = ""
    error: str | None = None
```

这段 StepExecutionResult

结构的关键，是它把“执行结果”从一段临时返回值，升级成了“下一步仍然能继续理解和消费的上下文对象”

。

一份中间结果如果只有 `rows`，它还只是数据；当它同时带上 `status`、`attempts`、`result_reference` 这些元信息时，它才真正变成 Agent 的“执行记忆”。

3.2 内存结果集方案

内存方案最适合当前课程这个阶段。

- 单机、单进程、单次分析链路
- 每一步结果量不大
- 目标是先让编排和容错逻辑跑通

实现非常直接：步骤成功后，把 `rows` 存到内存字典里，同时生成一个结果引用。

```
def _store_intermediate_result(
    self,
    step_id: str,
    columns: list[str],
    rows: list[dict[str, Any]],
):
    return self.result_store.put(step_id=step_id, columns=columns, rows=rows)
```

这里最关键的不是“打印出一段JSON”，而是执行器在真实运行时会为成功步骤生成稳定引用；后面第5节会用真实测试命令验证这个行为已经跑通。

3.3 临时表方案

当分析链路更长、结果集更大、或希望后续SQL直接复用前一步结果时，临时表会更合适。

它的核心价值在于：

- 可以直接被下一条SQL JOIN
- 结果结构更稳定，适合复杂聚合
- 更利于排查问题和追踪链路

但代价也很明确：

- 需要额外的建表、清理和命名规范
- 会引入数据库侧生命周期管理
- 跨数据库方言时实现复杂度更高

所以这节课的策略很明确：

这一课先把“结果能传下去、失败能处理掉”做扎实；临时表不是不重要，而是要等主链稳定后再把数据库侧工程复杂度接进来。

四、执行器升级：失败后到底怎么办

多步执行一旦进入真实数据库，失败是一定会发生的。问题不在于“能不能完全避免失败”，而在于失败之后系统行为是否可预测。

这节课实现了两条最核心的容错策略：

- `max_retries`：允许同一步骤重试若干次
- `failure_policy`：失败后选择 `abort` 或 `skip`

核心执行逻辑如下：

```
for attempt in range(1, self.max_retries + 2):
    raw_result = self.step_runner(question)
    normalized = StepExecutor._normalize_result(
        step=step,
        question=question,
        context_used=context_used,
        raw_result=raw_result,
        attempts=attempt,
        storage_backend=self.storage_backend,
    )
    if normalized.success:
        return normalized
```

如果依赖步骤已经失败，当前步骤不再盲目执行，而是直接进入 `skipped`：

```

if self._has_failed_dependency(step.depends_on, results_by_step):
    skipped = self.build_skipped_result(
        step=step,
        error="依赖步骤失败，当前步骤已跳过。"
    )

```

真正稳定的Agent，不是“每一步都保证成功”，而是每一步失败后都知道下一步该怎么处理。

五、代码实战

5.1 验证一：失败后重试并记录结果引用

```
uv run pytest tests/test_agent_planner.py::test_step_executor_retries_failed_step_and_records_result_reference -vv
```

输出（节选）：

```

===== test session starts =====
platform darwin -- Python 3.12.9, pytest-9.1.1, pluggy-1.6.0
cachedir: .pytest_cache
rootdir: /Users/will/Documents/will-obsidian/2-Projects/lector/企业级ChatBI系统/code/chatbi_mvp
configfile: pyproject.toml
plugins: langsmith-0.8.16, anyio-4.13.0
collected 1 item
tests/test_agent_planner.py::test_step_executor_retries_failed_step_and_records_result_reference PASSED [100%]
1 passed in 0.19s

```

这个测试验证了3件事：

- 第一步失败后会再次执行，而不是直接退出
- 成功结果会写成 `memory://step_1`
- `attempts` 会被正确记录为 2

5.2 验证二：依赖失败后跳过下游步骤

关键测试代码：`failure_policy="skip"`

```
uv run pytest tests/test_agent_planner.py::test_step_executor_skips_downstream_steps_after_failed_dependency -vv
```

输出（节选）：

```

===== test session starts =====
platform darwin -- Python 3.12.9, pytest-9.1.1, pluggy-1.6.0
cachedir: .pytest_cache
rootdir: /Users/will/Documents/will-obsidian/2-Projects/lector/企业级ChatBI系统/code/chatbi_mvp
configfile: pyproject.toml
plugins: langsmith-0.8.16, anyio-4.13.0
collected 1 item
tests/test_agent_planner.py::test_step_executor_skips_downstream_steps_after_failed_dependency PASSED [100%]
1 passed in 0.20s

```

这个测试验证了另外3件事：

- 首步失败后，第二步不会盲目执行
- 下游步骤状态会被标记成 `skipped`
- 跳过原因会明确写成“依赖步骤失败”

5.3 验证三：真实temp table存储与回读

```
uv run python - <<'PY'
from agent_planner import TempTableResultStore
store = TempTableResultStore()
try:
    ref = store.put('step_1', ['month', 'profit'], [{'month': '2026-05', 'profit': 920000}])
    rows = store.get(ref)
    print(ref)
    print(rows)
finally:
    store.cleanup()
PY
```

输出：

```
temp_table://tmp_agent_step_1
[{'month': '2026-05', 'profit': 920000}]
```

这条真实输出说明两件事：

- `temp_table` 后端会在 MySQL 会话里创建临时表
- 后续读取走的是 `TempTableResultStore.get(reference)`

六、典型误区

6.1 只传文本，不传状态

如果只把“上一步的一句摘要”传给后续步骤，而不记录成功、失败、跳过，后面的步骤就无法判断这份上下文是否可信。

6.2 把失败和跳过混为一谈

- `failed` 表示当前步骤真的执行了，但执行失败
- `skipped` 表示当前步骤没有执行，因为已经不满足前置条件

有些步骤失败后应该终止，有些步骤失败后只需要跳过依赖链路，如果两种场景都塞进一种策略里，系统会非常僵硬。

6.3 中间结果没有稳定引用

如果后续步骤只能“凭文字记忆”前一步结果，而没有这类明确引用，链路一长就很难排查问题。

memory://step_1

中间结果管理最怕的不是“实现复杂”，而是状态定义含糊。一旦边界不清楚，后面的报告生成和问题排查都会变得非常混乱。

success/failed/skipped

小结

这一课真正补上的，不是更多SQL，而是多步SQL背后的执行秩序。

- 第23课让系统学会“先计划，再执行”
- 第24课让系统学会“执行时记住发生了什么”
- 第25课才会基于这些稳定的中间结果，生成真正给业务看的分析报告

如果说第23课让Agent第一次“站起来”，那么第24课做的事情，就是让它开始连续走路而不是走一步停一步。

扩展思考

- 如果后续步骤希望直接JOIN前一步结果，临时表命名和清理策略应该怎么设计？
 - 如果一个步骤失败后，希望Agent自动改写SQL再试一次，本节课的执行器还需要补哪些角色？
-

附录

附录A：本节课对应代码

- code/chatbi_mvp/agent_planner.py
- code/chatbi_mvp/tests/test_agent_planner.py

附件（飞书无下载权限）：第 23 课：构建Plan-and-Execute Agent框架.pdf

附件（飞书无下载权限）：第 24 课：多SQL执行与中间结果管理.pdf